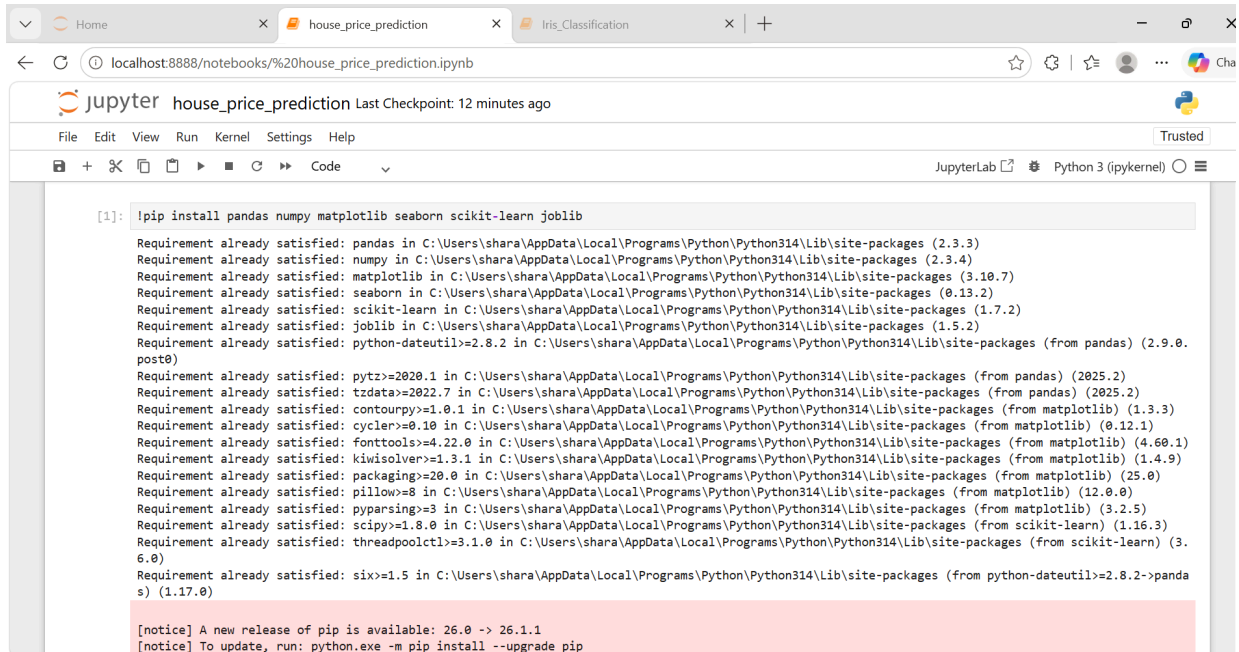


# 1. Install Libraries

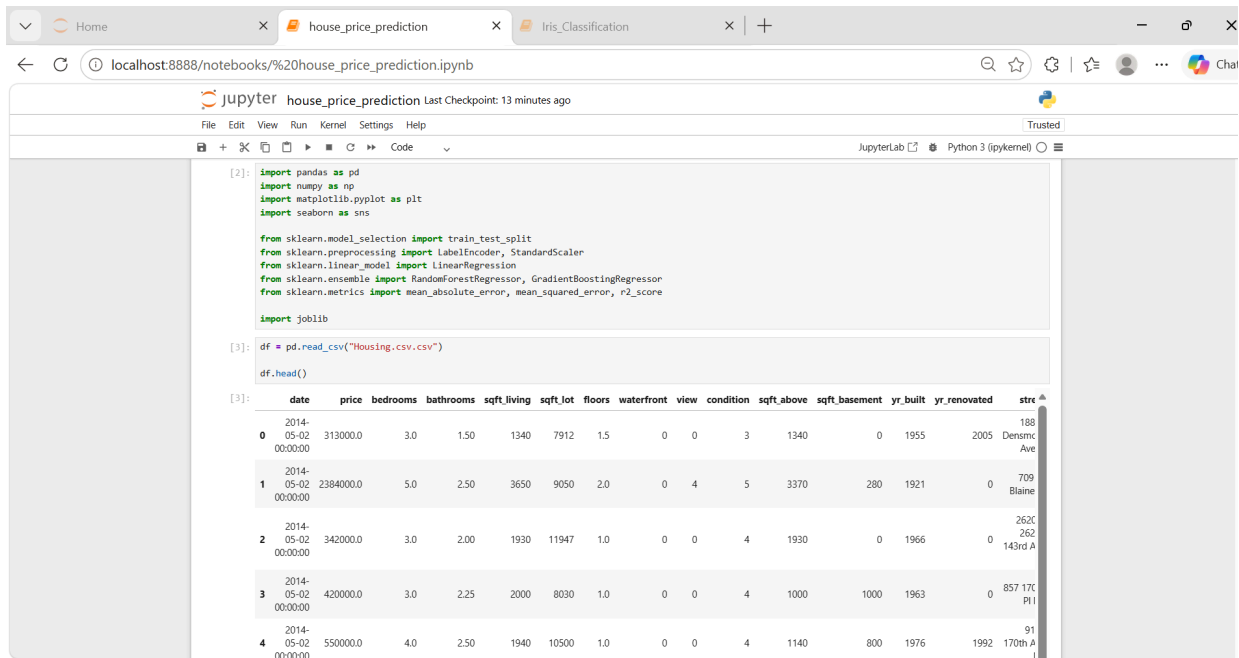


The screenshot shows a JupyterLab window with a browser at localhost:8888. The notebook is titled 'house\_price\_prediction' and shows the output of a pip install command. The output lists various requirements and their versions, all of which are already satisfied. A notice at the bottom indicates that a new release of pip is available (26.0 -> 26.1.1) and suggests running 'python.exe -m pip install --upgrade pip' to update it.

```
[1]: !pip install pandas numpy matplotlib seaborn scikit-learn joblib

Requirement already satisfied: pandas in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (2.3.3)
Requirement already satisfied: numpy in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (2.3.4)
Requirement already satisfied: matplotlib in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.10.7)
Requirement already satisfied: seaborn in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (0.13.2)
Requirement already satisfied: scikit-learn in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (1.7.2)
Requirement already satisfied: joblib in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (1.5.2)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (12.0.0)
Requirement already satisfied: pyparsing>=3 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: scipy>=1.8.0 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in C:\Users\shara\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

[notice] A new release of pip is available: 26.0 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```



The screenshot shows the same JupyterLab window. The notebook now shows the code to load the 'Housing.csv' dataset and a preview of the first few rows of the data. The data includes columns for date, price, bedrooms, bathrooms, sqft\_living, sqft\_lot, floors, waterfront, view, condition, sqft\_above, sqft\_basement, yr\_built, yr\_renovated, and street.

```
[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import joblib

[3]: df = pd.read_csv("Housing.csv.csv")
df.head()
```

|   | date                | price     | bedrooms | bathrooms | sqft_living | sqft_lot | floors | waterfront | view | condition | sqft_above | sqft_basement | yr_built | yr_renovated | street           |
|---|---------------------|-----------|----------|-----------|-------------|----------|--------|------------|------|-----------|------------|---------------|----------|--------------|------------------|
| 0 | 2014-05-02 00:00:00 | 313000.0  | 3.0      | 1.50      | 1340        | 7912     | 1.5    | 0          | 0    | 3         | 1340       | 0             | 1955     | 2005         | 188 Densmc Ave   |
| 1 | 2014-05-02 00:00:00 | 2384000.0 | 5.0      | 2.50      | 3650        | 9050     | 2.0    | 0          | 4    | 5         | 3370       | 280           | 1921     | 0            | 709 Blaine       |
| 2 | 2014-05-02 00:00:00 | 342000.0  | 3.0      | 2.00      | 1930        | 11947    | 1.0    | 0          | 0    | 4         | 1930       | 0             | 1966     | 0            | 262C 262 143rd A |
| 3 | 2014-05-02 00:00:00 | 420000.0  | 3.0      | 2.25      | 2000        | 8030     | 1.0    | 0          | 0    | 4         | 1000       | 1000          | 1963     | 0            | 857 17C Pl       |
| 4 | 2014-05-02 00:00:00 | 550000.0  | 4.0      | 2.50      | 1940        | 10500    | 1.0    | 0          | 0    | 4         | 1140       | 800           | 1976     | 1992         | 91 170th A       |

```

[4]: print(df.columns)

df.info()

df.isnull().sum()

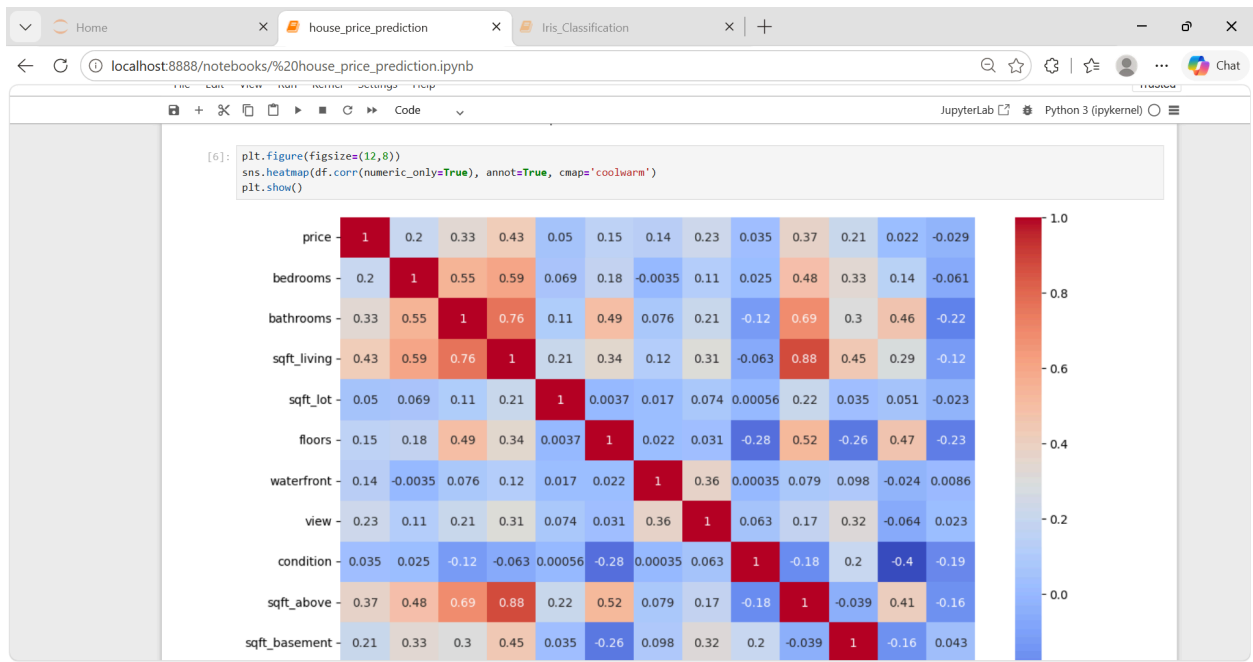
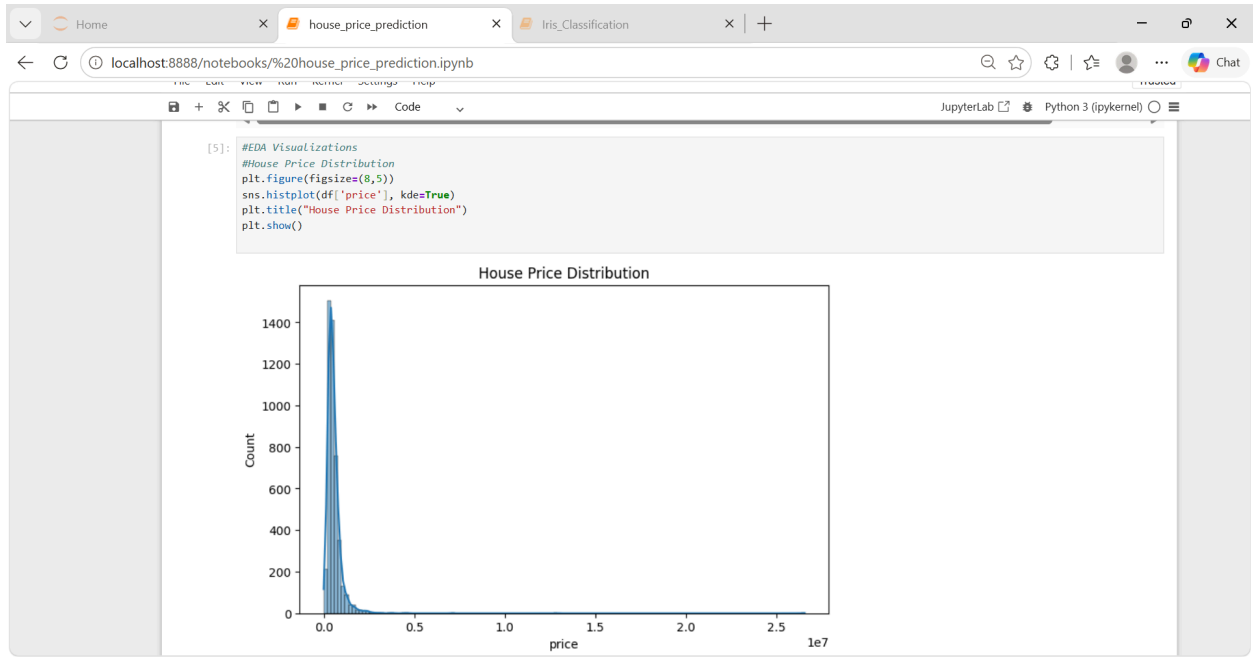
df.describe()

Index(['date', 'price', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
       'floors', 'waterfront', 'view', 'condition', 'sqft_above',
       'sqft_baseament', 'yr_built', 'yr_renovated', 'street', 'city',
       'statezip', 'country'],
      dtype='object')

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4600 entries, 0 to 4599
Data columns (total 18 columns):
 #   Column              Non-Null Count  Dtype  
---  --
 0   date                4600 non-null  object  
 1   price               4600 non-null  float64  
 2   bedrooms            4600 non-null  float64  
 3   bathrooms           4600 non-null  float64  
 4   sqft_living         4600 non-null  int64  
 5   sqft_lot            4600 non-null  int64  
 6   floors              4600 non-null  float64  
 7   waterfront          4600 non-null  int64  
 8   view                4600 non-null  int64  
 9   condition           4600 non-null  int64  
10  sqft_above          4600 non-null  int64  
11  sqft_baseament      4600 non-null  int64  
12  yr_built            4600 non-null  int64  
13  yr_renovated        4600 non-null  int64  
14  street              4600 non-null  object  
15  city                4600 non-null  object  
16  statezip            4600 non-null  object  
17  country             4600 non-null  object  
dtypes: float64(4), int64(9), object(5)
memory usage: 647.0+ KB

```

|       | price        | bedrooms    | bathrooms   | sqft_living  | sqft_lot     | floors      | waterfront  | view        | condition   | sqft_above  | sqft_baseament | yr_l        |
|-------|--------------|-------------|-------------|--------------|--------------|-------------|-------------|-------------|-------------|-------------|----------------|-------------|
| count | 4.600000e+03 | 4600.000000 | 4600.000000 | 4600.000000  | 4.600000e+03 | 4600.000000 | 4600.000000 | 4600.000000 | 4600.000000 | 4600.000000 | 4600.000000    | 4600.000000 |
| mean  | 5.519630e+05 | 3.400870    | 2.160815    | 2139.346957  | 1.485252e+04 | 1.512065    | 0.007174    | 0.240652    | 3.451739    | 1827.265435 | 312.081522     | 1970.781    |
| std   | 5.638347e+05 | 0.908848    | 0.783781    | 963.206916   | 3.588444e+04 | 0.538288    | 0.084404    | 0.778405    | 0.677230    | 862.168977  | 464.137228     | 29.731      |
| min   | 0.000000e+00 | 0.000000    | 0.000000    | 370.000000   | 6.380000e+02 | 1.000000    | 0.000000    | 0.000000    | 1.000000    | 370.000000  | 0.000000       | 1900.000    |
| 25%   | 3.228750e+05 | 3.000000    | 1.750000    | 1460.000000  | 5.000750e+03 | 1.000000    | 0.000000    | 0.000000    | 3.000000    | 1190.000000 | 0.000000       | 1951.000    |
| 50%   | 4.609435e+05 | 3.000000    | 2.250000    | 1980.000000  | 7.683000e+03 | 1.500000    | 0.000000    | 0.000000    | 3.000000    | 1590.000000 | 0.000000       | 1976.000    |
| 75%   | 6.549625e+05 | 4.000000    | 2.500000    | 2620.000000  | 1.100125e+04 | 2.000000    | 0.000000    | 0.000000    | 4.000000    | 2300.000000 | 610.000000     | 1997.000    |
| max   | 2.659000e+07 | 9.000000    | 8.000000    | 13540.000000 | 1.074218e+06 | 3.500000    | 1.000000    | 4.000000    | 5.000000    | 9410.000000 | 4820.000000    | 2014.000    |



```
localhost:8888/notebooks/%20house_price_prediction.ipynb

[7]: df.select_dtypes(include='object').columns
[7]: Index(['date', 'street', 'city', 'statezip', 'country'], dtype='object')

[8]: le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

[9]: #Handle Missing Values
df.fillna(df.mean(numeric_only=True), inplace=True)

[12]: # Convert price column to numeric
df['price'] = pd.to_numeric(df['price'], errors='coerce')

# Remove rows where price <= 0
df = df[df['price'] > 0]

# Apply log transformation safely
df['price'] = np.log1p(df['price'])

[13]: X = df.drop('price', axis=1)
y = df['price']

[14]: scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

[15]: X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

```
localhost:8888/notebooks/%20house_price_prediction.ipynb

[16]: lr = LinearRegression()
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)

[17]: #Random Forest
rf = RandomForestRegressor()
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

[18]: #Gradient Boosting
gb = GradientBoostingRegressor()
gb.fit(X_train, y_train)
gb_pred = gb.predict(X_test)

[19]: def evaluate_model(name, y_test, pred):
    mae = mean_absolute_error(y_test, pred)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)

    print(f"{name}")
    print(f"MAE:", mae)
    print(f"RMSE:", rmse)
    print(f"R2 Score:", r2)
    print("-"*30)

evaluate_model("Linear Regression", y_test, lr_pred)
evaluate_model("Random Forest", y_test, rf_pred)
evaluate_model("Gradient Boosting", y_test, gb_pred)
```

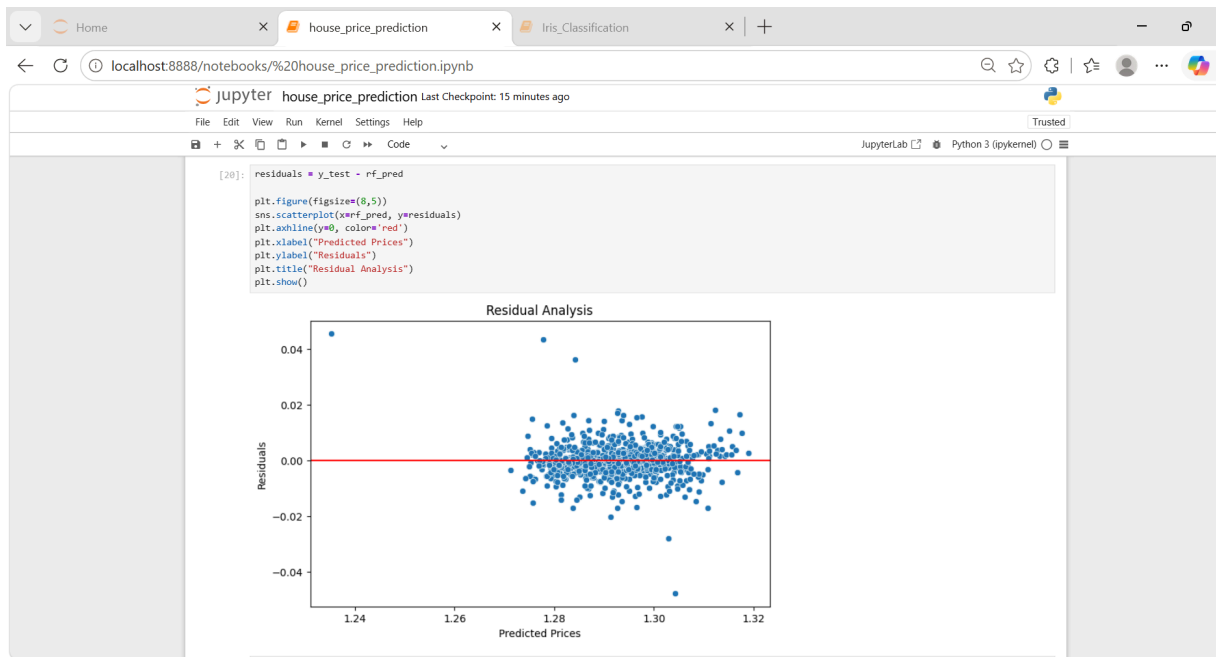
```
print(f"(name)")
print("MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)
print("-"*30)

evaluate_model("Linear Regression", y_test, lr_pred)
evaluate_model("Random Forest", y_test, rf_pred)
evaluate_model("Gradient Boosting", y_test, gb_pred)

Linear Regression
MAE: 0.005592545098614411
RMSE: 0.007437097914827301
R2 Score: 0.498313990457837
-----
Random Forest
MAE: 0.0038770002585275256
RMSE: 0.005862243553702828
R2 Score: 0.6882885847852912
-----
Gradient Boosting
MAE: 0.003866325330629238
RMSE: 0.005812710478978092
R2 Score: 0.6935339466927923
-----

[20]: residuals = y_test - rf_pred

plt.figure(figsize=(8,5))
sns.scatterplot(x=rf_pred, y=residuals)
plt.axhline(y=0, color='red')
plt.xlabel("Predicted Prices")
plt.ylabel("Residuals")
plt.title("Residual Analysis")
plt.show()
```



```
[21]: joblib.dump(rf, "house_price_model.pkl")
```

```
[21]: ['house_price_model.pkl']
```

```
[22]: #Prediction Example  
model = joblib.load("house_price_model.pkl")  
  
sample = [X.iloc[0]]  
  
sample_scaled = scaler.transform(sample)  
  
prediction = model.predict(sample_scaled)  
  
print("Predicted House Price:", np.exp(prediction))  
  
Predicted House Price: [3.61621424]
```